In [67]:
```
import json
import re
from pathlib import Path

import numpy as np
import pandas as pd

DATA_DIR = Path("data")
OUTPUT_DIR = Path("output")
```

In [68]:
```
# oppgave 1
# Her skal vi lese filen som 'bytes' og prøver å dekode den med UFT-8. Om
den ikke fungerer så går vi oveer til ISO-8859-1 som reserve.
# Funksjonen returnerer den dekodene teksten og tegnkodingen som ble brukt
```

In [69]:
```
def read_log_file(path) -> tuple[str, str]:
    """leser en loggfil og returnerer dens tekst og tegnkodingen som ble
brukt"""
    path = Path(path) #denne håndterer filstien
    raw = path.read_bytes() #denne leser filen uten å anta hvilken
tegnkoding som er brukt

    try:
        text = raw.decode("utf-8") #prøver å dekode filen med UTF-8
        encoding_used = "utf-8"
    except UnicodeDecodeError:
        text = raw.decode("iso-8859-1") #hvis UTF-8 ikke fungerer, bruker vi
ISO-8859-1 som reserve
        encoding_used = "iso-8859-1"

    return text, encoding_used #returnerer en tuple og to strenger
```

In [70]:
```
# her lærte jeg litt om regex som vi kalt inn som "re"
# ting den bruker her er feks (r"...") som betyr raw string som gjør at
python behandler (\) bokstavelig
# (^) betyr start på en linje, (\[)(\]) er klammeparantesene, (\d) betyr et
siffer far 0 te 9
# (\d{4}) betyr at det er nøyaktig 4 siffer, \s+ betyr at dte er flere eller
ett mellomrom
# (?P<timestamp>..) betyr at det den oppretter en navngitt gruppe som henter
tidspunkt, og (|) betyr "eller"

#vi gjør dette slikt at python kan lese en loggfil, og kontrollere at
linjene har riktig format og mønster og for å automatisere det. Uten dette
så hadde det tatt lang ting å lese alt

LOG_PATTERN = re.compile(
    r"^\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\]\s+"
    r"\[(?P<register_id>[^\]]+)\]\s+"
    r"item=(?P<item>\S+)\s+" #i filen generate_data_for_d2, så står det
ITEMS og ikke item, men det akn vi ikke kalle disse fordi ITEMS er pyhton
lista med alle mulige varer osv, mens iten er avnet på feltet regexen henter
ut for hver enkelt logglinje
    r"price=(?P<price>\d+(?:\.\d+)?)\s+"
    r"(?P<currency>[A-Z]{3})\s+"
    r"\((?P<status>SUCCESS|REFUND|CANCELLED)\)\s*$",
    re.MULTILINE,
)
```

In [71]:
```
def parse_log_text(text: str) -> list[dict]:
    """henter ut gyldige salgslinjer som en liste med dictionary på 6
felt"""
    records = [] #lager tom liste som skla innholde lalle gyldige
salgslinjer vi finner
    for m in LOG_PATTERN.finditer(text): #her finner vi de gyldigere
salgslinjene, der LOG_PATTERN er regex mønsteret vi lagde før, og
finditer(text) finner alle treff i teksten som passer møsnteret over
        rec = m.groupdict()
        rec["price"] = float(rec["price"])
        records.append(rec)
    return records
```

In [72]:
```
# oppgave 2
```

In [73]:
```
def find_log_files(directory, extension: str = ".log") -> list[Path]:
    """finner alle filer med den gitte filtypen, også i undermappene """
    directory = Path(directory)
    found: list[Path] = [] # her lsger jeg en tom liste som skal innholder alle de filene vi finner
    for item in directory.iterdir(): # iterdir() går gjennom alle ting som direkte ligger i mappen(både disse filene og undermappene vi har)
        if item.is_dir(): # generelt her så så sier den at om elementet er en mappe, så kaller funskjonene på seg selv for å undersøke innholdet den har i seg
            found.extend(find_log_files(item, extension)) #den find_log_files ... leter viderei undermappen, og extend() legegr til alle de filene som ble funnet i den listen fra tidligere
        elif item.is_file() and item.suffix == extension: #denne sier at om elementet ikke er en mappe men en fil, så undersøker den om filtypen stemmer emd .log
            found.append(item) # hvilken den stemmer, så legger append() filstien til listen
    return found
```

In [74]:
```
def get_active_registers(node) -> set[str]: # node er delen av JSON dataene som funskjoenn undersøker
    """samler inn ID-ene til alle kassesystemene der active er True"""
    active: set[str] = set() #her lager jeg et tomt sett som skal samle ID til de aktive kassesystemene, og spesifikt set() er for å ungå dupliserte IDer
    if isinstance(node, dict): #her skjekker den om hvilke typer verdi noden innholder, om det er en dictinary osv.
        if node.get("active") is True and "id" in node: # node.get("active") heter ut verdien av nøkklen active, og skjekker om verdien faktisk er true, og om den innholder en id
            active.add(str(node["id"])) # her legger den id til settet som string
        for value in node.values(): # denne gjør at funskjonen undersøker alle verdiene i dictionarien, også de som har nye dictionaries eller lister inni seg
            active |= get_active_registers(value) # |= slår sammen settet som returneres med settet "active"
    elif isinstance(node, list): #denne sier at om likevell noden er en liste så går funskjonene gjennom hvert element og undersøker på samme måte
        for element in node:
            active |= get_active_registers(element)
    return active
```

In [75]:
```
# oppgave 3
```

In [76]:
```
def remove_price_outliers(price) -> np.ndarray:
    """erstatter verdier med |x-gjennomsnitt▷ 3* standardavvik av medianene til de gyldige verdiene """
    arr = np.asarray(price, dtype=float).ravel() # numpy gjør prisene til en numpy array, og den dtype=float gjør at verdiene behandles som desimaltall, i tilleg gjør ravel() om arrayen til en dimesjon
    if arr.size == 0: #denne skjekker om listen er tom, og om den er det så returnerer funskjonene den tomme arrayen med en gang
        return arr
    mean, std = arr.mean(), arr.std() #denne regner ut gjennomsnittet og standardavviket(sier vhro mye prisene varierer rundt gjennomsnittet)
    outlier_mask = np.abs(arr - mean) > 3 * std # denne linjer skjekker om hver pris ligger mer en 3 standardavvik far gjennomsnittet, og resultaatet er en maske med True for unormale priser(de over 3 SD), og false for de andre
    valid = arr[~outlier_mask] # denne ~ bare snur masken slikt t vi velger prisene som ikke er markert som unormale
    if valid.size == 0: #skjekker om det finner gyldige priser, der om ingen er det så returnerer den arrayen uten å endre noe,
        return arr
    cleaned = arr.copy() #så erstatter den de unormale prisene med medianen, men dette gjøres til en kopi og ikke orgnial arrayen, vi gjer det for unormale verdier fordi de ikke er representive for helheten, så for at tolkningen vår ikke blir påvirket av dem ekremverdieene eller outliere så bare setter vi inn medianen
    cleaned[outlier_mask] = np.median(valid)
    return cleaned
```

In [77]:
```
def compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray: #sales-count er antall salg, og refund_count er antall refund. np.ndarray er bare at det forventes verdienee skal være numpy arrays
    """refusjonsandel(andel salg som blir refundert) beregnet elementvis, der epsilon hindrer divisjon med null"""
    epsilon = 1e-6 #her lager jeg bare en liten verdi, så den ikke blir null, fordi da går det ikke an å dele
    sales_count = np.asarray(sales_count, dtype=float) # i de to linjene her så bare gjøres verdiene om til desimaltall
    refund_count = np.asarray(refund_count, dtype=float)
    return refund_count / (sales_count + epsilon) #beregner refusjonsandelen og returnerer det
```

In [78]:
```
#oppgave 4
```

In [79]:
```
# denne funskjonen gjør listen med dictionarien om til en tabell, og
soerterer dem kronologisk fra tidspunktet
def build_dataframe(records: list[dict])→ pd.DataFrame:
    """gjør om records til en dataframe med dato og klokkeslett som indeks,
sortert i en kronologisk rekkefølge"""
    df = pd.DataFrame(records)
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp").sort_index()
    return df

# denne funksjonen grupperer dataene etter vare og beregner summen av price
i intervaller på en time
def hourly_revenue(df: pd.DataFrame) → pd.Series:
    """total pris per vare på 1 timers intervall"""
    return df.groupby("item")["price"].resample("1h").sum()

# denne funskjonen lager en tabell der hver vare får si egen rad, og hver
status for sin egen kolonne 8 latså sammenligner omsetning per vare og
status
def status_pivot(df: pd.DataFrame) → pd.DataFrame:
    """total omsetning per vare på tvers av de ulike statuskategoriene"""
    pivot = pd.pivot_table(
        df, values="price", index="item", columns="status", aggfunc="sum",
fill_value=0.0,
    )
    return pivot.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"],
fill_value=0.0)

# denne funskjonenberegner refusjonsandelen per vare, altså at den teller
hvor mange salg og refusjoner hver vare har og beregner forholdet mellom
dem)
def item_refund_ratios(df: pd.DataFrame) → pd.DataFrame:
    """refund ratio per vare der SUCCESS = sales count, og REFUND = refund
count"""
    counts = df.groupby(["item", "status"]).size().unstack(fill_value=0)
    counts = counts.reindex(columns=["SUCCESS", "REFUND", "CANCELLED"],
fill_value=0)
    ratio = compute_refund_ratio(counts["SUCCESS"].to_numpy(),
counts["REFUND"].to_numpy())
    return pd.DataFrame(
    {"sales_count": counts["SUCCESS"], "refund_count": counts["REFUND"],
"refund_ratio": ratio},
    index=counts.index,
)
```

In [80]:
```
# oppgave 5
```

In [ ]:
```
#denne bare starter hovedfunksjonen
def main(data_dir: Path = DATA_DIR, output_dir: Path = OUTPUT_DIR) →
pd.DataFrame:
    output_dir.mkdir(parents=True, exist_ok=True)
```

```
    #1. her finner vi loggfiler og aktive registere
    log_files = find_log_files(data_dir / "2026")
    with open(data_dir / "store_config.json", encoding="utf-8") as f:
        active_registers = get_active_registers(json.load(f))
    print(f"Fant {len(log_files)} loggfiler. Aktive registre:
{sorted(active_registers)}")

    #2. denne leser alle loggfilene og henter de gyldige salgslinjene ut i
en liste og fjerne transaksjoner som kommer fra kassesystemer som ikek er
aktive
    records: list[dict] = []
    for path in sorted(log_files):
        text, encoding = read_log_file(path)
        parsed = parse_log_text(text)
        print(f"   {path.name}  ({path.parent.name}): {encoding},
{len(parsed)} gyldige linjer")
        records.extend(parsed)
    records = [r for r in records if r["register_id"] in active_registers]

    #3. her gjøres bare postene om til en dataframe
    df = build_dataframe(records)

    #4. her fjernes unormale priser som ikek representerer helheten
    df["price"] = remove_price_outliers(df["price"].to_numpy())

    #5.denne linjen ganger alle priser med valuta NOK med 10 som oppgaven
sier
    df.loc[df["currency"] == "NOK", "price"] *= 10.0

    #6. her bruer vi de tidligere definerte funskjonene der hourly beregner
prisene per vare og time, status summerer beløpene per vare fordelt på
status, og item teller salg og fefunds per vare og beregner refunden
    hourly = hourly_revenue(df)
    pivot = status_pivot(df)
    ratios = item_refund_ratios(df)

    #7. lagrer resultatene i filer i output_dir
    df.to_csv(output_dir / "cleaned_transactions.csv")
    hourly_dict = {
        item: {ts.strftime("%Y-%m-%d %H:%M:%S"): round(float(v), 2)
               for ts, v in group.droplevel("item").items()}
        for item, group in hourly.groupby(level="item")
    }
    with open(output_dir / "hourly_revenue.json", "w", encoding="utf-8") as
f:
        json.dump(hourly_dict, f, indent=2)
    pivot.to_csv(output_dir / "status_pivot_table.csv")
    ratios.to_csv(output_dir / "item_refund_ratios.csv")

    print("\nStatus-pivot: \n", pivot.round(2))
    print("\nRefund ratios: \n", ratios.round(4))
```

```
    return df



df_clean = main()
df_clean.head()
```

```
Fant 9 loggfiler. Aktive registre: ['POS_01', 'POS_02', 'POS_03']
   register_sales.log  (POS_01): utf-8, 80 gyldige linjer
   register_sales.log  (POS_02): utf-8, 80 gyldige linjer
   register_sales.log  (POS_03): utf-8, 80 gyldige linjer
   register_sales.log  (POS_01): utf-8, 80 gyldige linjer
   register_sales.log  (POS_02): utf-8, 80 gyldige linjer
   register_sales.log  (POS_03): utf-8, 80 gyldige linjer
   register_sales.log  (POS_01): utf-8, 80 gyldige linjer
   register_sales.log  (POS_02): utf-8, 80 gyldige linjer
   register_sales.log  (POS_03): utf-8, 80 gyldige linjer

Status-pivot:
 status      SUCCESS  REFUND  CANCELLED
item
Cappuccino  1549.91  307.41     213.55
Croissant   1033.57  382.72     249.85
Espresso     948.77  169.05     234.81
Latte       1043.55  401.00     325.24
Muffin      1181.10  246.63     189.53
Sandwich    2960.01  558.16     402.78

Refund ratios:
            sales_count  refund_count  refund_ratio
item
Cappuccino           81            21        0.2593
Croissant            77            21        0.2727
Espresso             82            20        0.2439
Latte                61            19        0.3115
Muffin               80            25        0.3125
Sandwich             92            22        0.2391
```

Out[81]:

| timestamp | register_id | item | price | currency | status |
|---|---|---|---|---|---|
| **2026-10-15 07:00:42** | POS_02 | Muffin | 3.01 | USD | SUCCESS |
| **2026-10-15 07:00:58** | POS_03 | Muffin | 3.81 | USD | SUCCESS |
| **2026-10-15 07:01:13** | POS_01 | Sandwich | 72.00 | NOK | SUCCESS |
| **2026-10-15 07:01:19** | POS_02 | Muffin | 3.81 | USD | SUCCESS |
| **2026-10-15 07:01:32** | POS_03 | Espresso | 2.71 | USD | CANCELLED |